

Instituto Tecnológico de Morelia
Examen Diagnóstico - Fundamentos de Ingeniería de Software

Materia: Tópicos Selectos de Tecnologías Web y Móvil

Profesor: Jesús Eduardo Alcaraz Chávez

Nombre del alumno:

Cristian Mercado Martin

Fecha: 27/08/26

Instrucciones: Responde de manera clara y concisa a cada una de las siguientes preguntas abiertas. El propósito de esta evaluación es medir tus conocimientos previos en Ingeniería de Software.

1. Metodologías: ¿Cuál es la diferencia principal entre una metodología de desarrollo tradicional (como Cascada) y una metodología ágil (como Scrum) frente a los cambios en los requisitos?

La diferencia radica en la manera en que se estructuran los equipos, en el desarrollo tradicional tenemos un equipo completo (dividido en departamentos dependiendo el tamaño del grupo) trabajando en un mismo objetivo o aplicación completa, en cambio ágil crea grupos pequeños pero con la suficiente autonomía para crear productos entregables y pequeños a comparación del tradicional, además de que aquí predomina la filosofía: desarrollo semanal, testeo y retroalimentación.

2. Requerimientos: Explica la diferencia entre requerimientos funcionales y no funcionales, dando un ejemplo de cada uno aplicable a una plataforma web.

Los requerimientos funcionales son aquellos que representan las cualidades o características de un producto o web, por ejemplo el login, gestión de personal o clientes. En cambio los no funcionales tienen que ver más con la experiencia de usuario representados en la distribución de UI, seguridad del software, que tan intuitivo es o qué tan rápido funciona.

3. Arquitectura: Describe el modelo Cliente-Servidor y explica brevemente cómo se comunican el frontend y el backend en una aplicación web moderna.

Este modelo se basa principalmente en peticiones, se crea un intermediario llamado API donde el usuario tiene una interacción con el sistema backend mediante el uso de métodos tales como GET, POST, DELETE, etc. donde se enviará o recibirá información.

Puede ejemplificarse con la analogía del restaurante, donde el restaurante es el front, el cliente no puede interactuar directamente con el cocinero (backend) así que necesita hacerle peticiones al mesero (API) para poder realizar su pedido.

4. Bases de Datos: ¿En qué escenarios recomendarías utilizar una base de datos relacional (SQL) frente a una no relacional (NoSQL) para el almacenamiento de datos en una aplicación?

La base de datos relacional es más usada cuando ocupamos una base robusta e inmutable, además es la que se suele usar en proyectos escalables o de gran magnitud ya que nos aseguramos que todos los datos de un objeto son idénticos y consistentes ya que comparten las mismas columnas .

En cambio cuando ocupamos una no relacional suele ser en casos donde necesitamos un desarrollo más flexible ya que no tenemos atributos fijos sino simplemente es una colección de datos agrupados en formato JSON y se suele usar en proyectos no gran magnitud

5. APIs: ¿Qué es una API REST y qué papel fundamental juega en la integración entre una aplicación móvil y los servidores (backend)?

La api rest es el intermediario en la comunicación de dos sistemas, mediante el uso de métodos como GET, POST, DELETE, PATCH puede lograr la comunicación de lo que ve el usuario con lo que hay dentro de la base de datos

6. Control de Versiones: Explica la importancia de utilizar Git en un equipo de desarrollo de software y describe brevemente qué es un "merge conflict" (conflicto de fusión).

La importancia radica en las facilidades para trabajar en equipo, ya que cada desarrollador puede trabajar en ramas/features de manera independiente y además

ofrece facilidades para fusionar el trabajo de los colaboradores y verificar que todos este bien, por ejemplo con el uso de workflows de GITHUB para correr los test de manera automática, además en usar Git permite retroceder fácilmente en caso de que algo haya mal a una versión que si funciona correctamente.

Un merge conflict es cuando dos colaboradores editaron un mismo archivo y partieron desde el mismo commit entonces GIT no sabe cual de las dos versiones es la correcta, asi que toca que editar o hacer una selección manual.

7. Pruebas: ¿Qué son las pruebas unitarias (unit testing) y por qué son cruciales para asegurar la calidad del software antes de su paso a producción?

Las pruebas unitarias, son aquellas que ponen a prueba el correcto funcionamiento de alguna característica de nuestro software, NO testea la aplicacion completa, solo una funcion como por ejemplo CREAR USUARIO. Son cruciales al momento del desarrollo de alguna feature para ver su funcionamiento en un entorno controlado

8. POO: Define los conceptos de encapsulamiento y polimorfismo de la Programación Orientada a Objetos, y menciona cómo ayudan a crear un código más mantenible.

Encapsulamiento se refiere a hacer privado algún atributo de una clase, en resumen sirve para proteger información y controlar el acceso a esta dentro de un sistema, en cambio el polimorfismo se refiere a como una clase puede tener varias formas o distintas formas de comportarse dentro de un sistema. Ayudan principalmente ayuda a tener una estructura más organizada y más legible al dar claridad a la información sensible y relacionar clases.

9. Patrones de Diseño: ¿Qué es el patrón de arquitectura Modelo-Vista-Controlador (MVC) y cómo ayuda a organizar el código en el desarrollo de software?

Se refiere a la forma en que esta organizado nuestro software, digamos que separamos lo relacionado a lo que interactúa con la base de datos con lo que interactúa con el usuario a nivel de código. La vista son por ejemplo el codigo en HTML que le llega al usuario, los Modelos es la definición de clases en nuestra base de datos y el controlador es el intermediario entre ambas partes.

Ayuda en que facilita el mantenimiento, ya que en caso de que haya un error es más fácil rastrear el problema, es más legible al tener todo el código seccionado y dividido en archivos no tan complejos.

10. Seguridad: Explica la diferencia técnica entre "autenticación" y "autorización" en el contexto de seguridad de una aplicación.

Autenticación es decir QUIEN eres a una aplicación, es decir si eres un alumno, un profesor o algún administrador, y por lo tanto saber a qué partes de nuestro software direccionarte.

En cambio autorización es decirle al sistema A QUE puedes acceder dentro del sistema aquí establecemos que información o menús puede consultar el usuario,